PIET NON ASMP – 04

Loki — A Transparent Git-Style Version Control System



Architecture, Internal Workflows, Storage Design, and Installation

An educational, local Git-style version control system implemented in Go.

Ajay Agnihotri	2303031240181	**Animesh Singh Rajwar**	2303031240205
Nilesh Saraswat	2303031240715	**Vedant Mahale**	2303031241475

Parul Institute of Engineering and Technology (PIET) · Parul University, Vadodara, Gujarat, India

B.Tech. · Computer Science and Engineering (Specialization in Artificial Intelligence)

Project Guide: Prof. Rahul Moud, Assistant Professor · Academic Year 2026–2027

TEAM · PIET NON ASMP – 04

Project Team and Academic Details

Member 1

Ajay Agnihotri

Enrollment No.

2303031240181

Member 2

Animesh Singh Rajwar

Enrollment No.

2303031240205

Member 3

Nilesh Saraswat

Enrollment No.

2303031240715

Member 4

Vedant Mahale

Enrollment No.

2303031241475

Team / Group: PIET NON ASMP – 04

College: Parul Institute of Engineering and Technology (PIET)

University: Parul University, Vadodara, Gujarat, India

Project Guide: Prof. Rahul Moud, Assistant Professor

Department: Computer Science and Engineering (Specialization in Artificial Intelligence)

Degree: Bachelor of Technology (B.Tech.)

Academic Semester: 7th Semester

Academic Year: 2026–2027

Agenda

01 **Introduction to version control**
Slide 4

02 **Loki overview and capabilities**
Slides 5–7

03 **System architecture**
Slides 8–13

04 **Object model and storage**
Slides 14–24

05 **Command workflows**
Slides 25–28

06 **Branching and merging**
Slides 29–34

07 **Configuration, release and installation**
Slides 35–38

08 **Limitations and future scope**
Slides 39–40

What Is Version Control?

Commit

A saved snapshot of the project, with a message.

History

The ordered record of those snapshots over time.

Branch

A parallel line of work that can be merged back later.

Illustrative example: a small website project

`main`

v1
Create index.html

→

v2
Add login page

→

v3
Fix a typo

`feature branch (starts from v2)`

v2b
Dark-mode experiment

General version-control concepts only. Loki-specific behavior starts on the next slide.

A · INTRODUCTION AND OVERVIEW

Loki is an educational, local, Git-style version control system written in Go.

It demonstrates core version-control concepts: object storage, staging, commits, branches, tags, diffs, merging and reverting.

LANGUAGE	SCOPE	PURPOSE	COMPATIBILITY
Go	Local repositories only	Educational implementation	Git-inspired, not fully Git-compatible

Project Objectives

Repository initialization

Create a local repository and its .loki metadata.

Staging

Snapshot files as blobs and record them in an index.

Commits

Turn staged content into commit and tree objects.

Object storage

Store objects by SHA-1 identifier, compressed with zlib.

History and references

Branches, tags and the HEAD reference.

Diff, merge and revert

Compare, combine and undo changes within one local repository.

Summarized from the architecture documentation; the written project report may phrase the objectives differently.

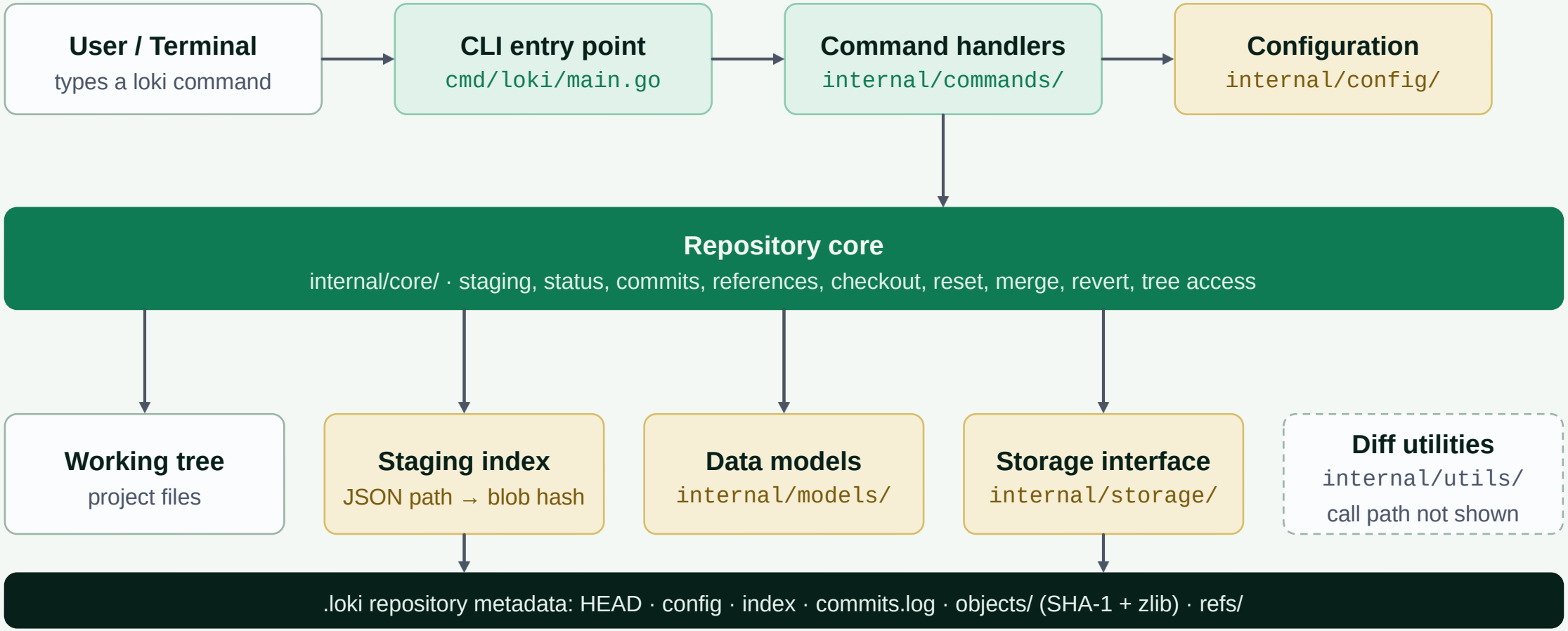
Key Features: Implemented Commands

`loki init` Initialize a repository in the current directory	**`loki add`** Stage one or more files for commit	**`loki status`** Show tracked, staged, modified, deleted and untracked files	**`loki diff`** Working tree vs index; --staged: index vs HEAD	**`loki commit`** Commit the staged snapshot with author metadata (-m)
`loki log` Show the commit history stored in the repository	**`loki branch`** Create or list branches	**`loki checkout`** Switch to a branch, tag or commit reference	**`loki merge`** Merge a branch: fast-forward and 3-way flows	**`loki reset`** Reset HEAD, index or working tree to a target
`loki rm` Remove files from the index and working tree	**`loki revert`** New commit that reverses an earlier commit	**`loki tag`** Create, list, delete or annotate tags	**`loki config`** Read or write repository configuration values	**`loki help`** Print usage for the CLI and its commands

All 15 are marked implemented in Loki's documentation. Local only: no fetch, pull or push.

B · SYSTEM ARCHITECTURE

High-Level Architecture



B · SYSTEM ARCHITECTURE

Architectural Layers

CLI entry point `cmd/loki/`	Entry point of the program; dispatches each command to its handler.
Command handlers `internal/commands/`	Parse arguments and present command output.
Repository core `internal/core/`	Coordinates staging, status, commits, references, checkout, reset, merge, revert and tree access.
Data models `internal/models/`	Serialize and parse Blob, Tree, Commit and Tag objects.
Storage, config, utilities `internal/storage/ config/ utils/`	Store and read objects (SHA-1 + zlib); read and write configuration and find the repository root; generate text-line diffs.

CLI Entry Point and Command Handlers

$ loki status

A command enters the program:

- `cmd/loki/main.go`
- `internal/commands/`
- `internal/core/`

Execution is traced only as far as the documentation goes.

CLI entry point

`cmd/loki/main.go`

Where every command starts. It dispatches the command to the matching handler.

Command handlers

`internal/commands/`

Parse arguments and present output for commands such as init, add, status, diff, commit, log, branch, checkout, merge, reset, rm, revert, tag, config and help.

The source material does not show per-command execution paths, so the paths are not assumed to be identical.

B · SYSTEM ARCHITECTURE

Core Repository Logic

Repository core

`internal/core/`

Coordinates every repository operation and works with:

Staging index · Data models · Storage interface · Working tree



- **Staging**
- **Status**
- **Commits**
- **References**
- **Checkout**
- **Reset**
- **Merge**
- **Revert**
- **Tree access**

References cover branches, tags and HEAD. Merging is local only.

B · SYSTEM ARCHITECTURE

Models, Storage and Utilities

Data models

`internal/models/`

Serialize and parse the four object types: Blob, Tree, Commit and Tag.

Storage

`internal/storage/`

Stores and reads serialized objects using SHA-1 identifiers and zlib compression, behind an object-storage interface.

Config and diff

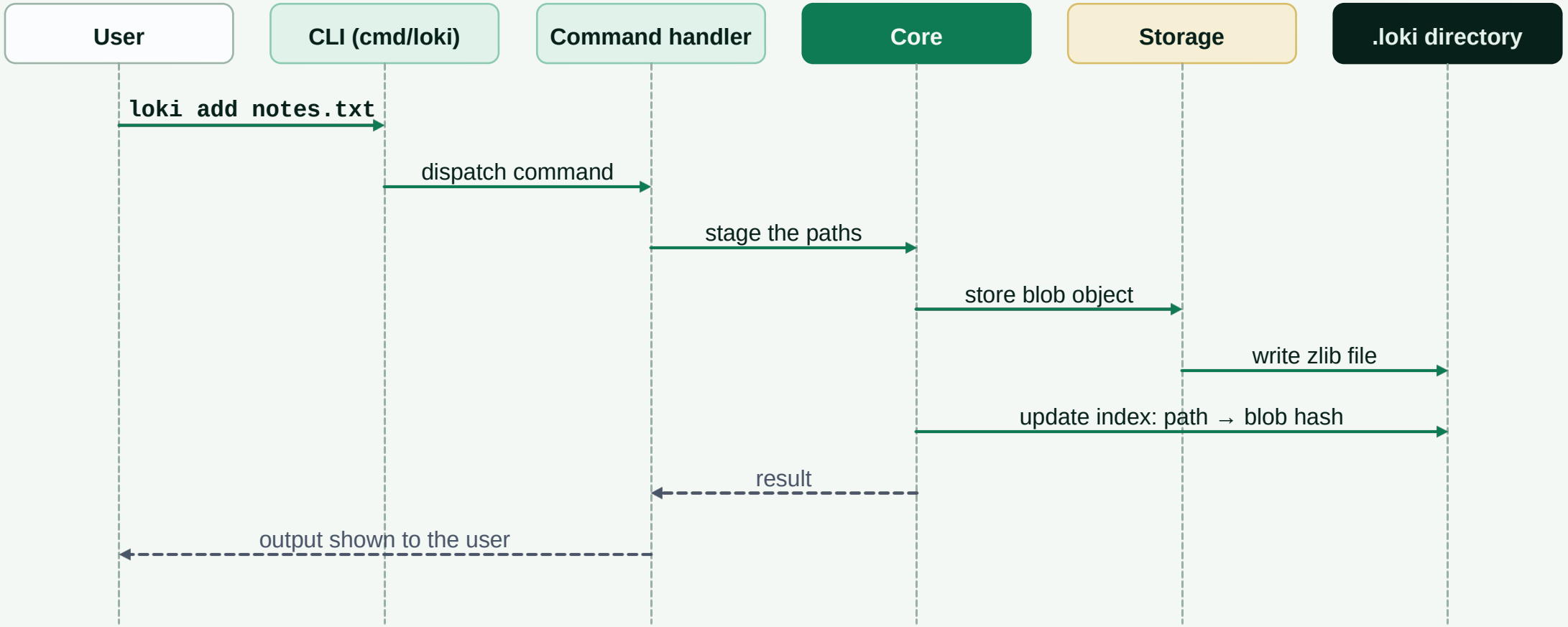
`internal/config/ · internal/utils/`

Read and write configuration and discover repository roots; generate and format text-line diffs.

HOW MODELS AND STORAGE INTERACT

Model serializes the object → **Storage hashes the bytes (SHA-1)** → **Storage compresses (zlib)** → **Written under .loki/objects/**

End-to-End Command Flow: loki add



Repository Initialization

BEFORE LOKI INIT (ILLUSTRATIVE)

```
project/
└── your files
```

AFTER LOKI INIT (ILLUSTRATIVE)

```
project/
├── your files
└── .loki/   ← created by loki init
```

What it does

Initializes a new Loki repository in the current working directory.

Role of .loki

Holds the repository metadata: HEAD, config, index, objects and references (next slide).

Finding the repository

internal/config/ reads and writes configuration and discovers repository roots.

Which .loki entries exist immediately after init is not detailed in the supplied architecture summary.

Internal .loki Directory Structure

```
project/
├── .lokiignore            # optional ignore patterns
└── .loki/
    ├── HEAD               # current branch or detached commit
    ├── config             # repository configuration
    ├── index              # JSON: path -> blob hash
    ├── commits.log        # append-only log for loki log
    ├── objects/          # compressed serialized objects
    └── refs/
        ├── heads/        # local branch references
        └── tags/         # tag references
```

Cyan = directory · White = file

.lokiignore

Optional ignore patterns at the project root, outside .loki/.

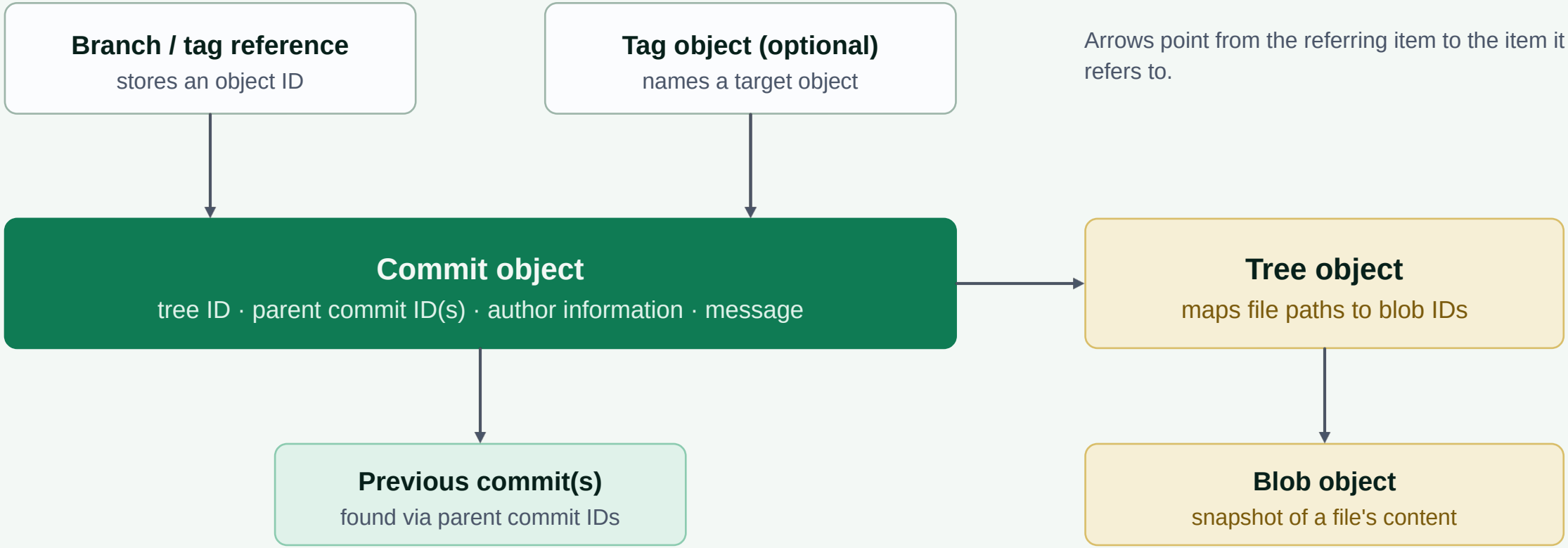
Temporary state

Merge and revert may create temporary state files while conflicts are resolved. Their names are not given in the summary.

objects/ layout

Folder name = first two hash characters (slide 21).

Loki Data Model



Blob Objects

1 · WORKING FILE

notes.txt

```
Hello Loki
```

Illustrative content

→

2 · BLOB OBJECT

Snapshot of the content

Serialized by the Blob model.

→

3 · STORED OBJECT

.loki/objects/

Named by its SHA-1 hash, compressed with zlib.

The blob hash

SHA-1 over the complete serialized bytes is the object ID.

Same bytes, same ID

Identical serialized bytes give the same object ID in the normal write path.

Paths live elsewhere

Trees and the index map file paths to blob IDs.

Tree Objects

Tree object: path → blob ID

illustrative paths



As documented

A tree maps file paths to blob IDs.

Used by commits

A commit records the ID of one tree.

Not drawn

Nested per-directory tree objects are not described in the documentation.

Commit Objects

A commit records

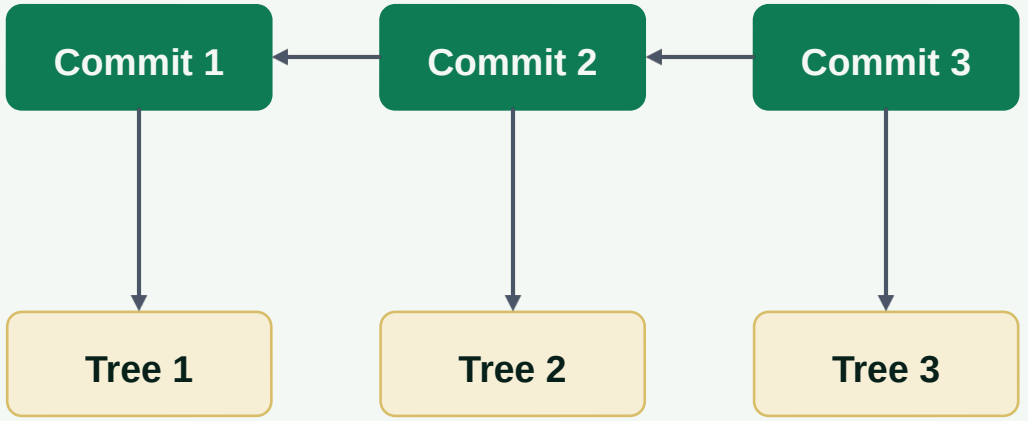
Tree ID
the snapshot of staged content

Parent commit ID(s)
links to the previous commit(s)

Author information

Commit message

ILLUSTRATIVE HISTORY



Left arrows: parent links. Down arrows: tree ID. Each commit stands for one whole-project snapshot.

Tags and References

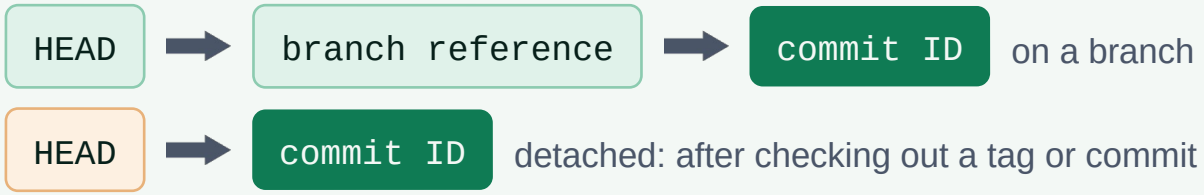
`HEAD`
Identifies the current branch, or a detached commit.

`refs/heads/`
Local branch references; each stores an object ID.

`refs/tags/`
Tag references, also storing object IDs.

Tag objects
An annotated tag is an object naming a target; a lightweight tag is a plain reference.

HOW HEAD IDENTIFIES A STATE (CONCEPTUAL)



Content-Addressed Object Storage

Serialized object bytes → SHA-1 over the complete bytes → Object ID (40 hex characters)

STORAGE PATH PATTERN

```
.loki/objects/<first-2-hex>/<remaining-38-hex>
```

The ID is the location
The hash tells Loki where to find the object.

Not encryption
Content addressing names data; it does not hide or authenticate it.

SHA-1 as an identifier
Not a modern collision-resistance guarantee.

Object Serialization and Compression

WRITING AN OBJECT, IN THE DOCUMENTED ORDER

1
Serialize
The model turns the object into bytes.

2
Hash
SHA-1 over the complete serialized bytes gives the object ID.

3
Compress
zlib compresses the bytes for storage.

4
Store
Written under .loki/objects/ using the ID as the path.

ID is independent of compression
Compression changes the disk representation, not the object ID.

Reading an object
Objects are decompressed, but the hash is not recomputed to verify the ID.

The Staging Area and Index

Working file
`loki add README.md`

→

Blob object
stored in .loki/objects/

→

Index entry
path → blob hash in .loki/index

ILLUSTRATIVE .LOKI/INDEX (JSON)

```
{
  "README.md": "<blob hash>"
}
```

A custom JSON mapping, not Git's index format.

Staging

loki add stores file snapshots as blobs and updates the index.

Committing is separate

loki commit creates a commit from the staged content.

Commit History Representation

WHAT LOKI LOG READS

`.loki/commits.log`

An append-only log of recorded commits.

```
entry 1
entry 2
new commit → appended at the end
```

WHAT LOKI LOG DOES NOT DO

Walk commit ancestry from HEAD

It does not follow parent commit IDs backwards through the history graph.

`HEAD` → `commit` → `parent`

Not used by loki log.

Commit objects still record parent commit IDs; loki log simply does not use them to build its output.

Basic Loki Workflow

```
$ loki init
$ echo "hello" > notes.txt
$ loki add notes.txt
$ loki status
$ loki diff
$ loki commit -m "Add notes"
$ loki log
```

1. Initialize the repository
2. Create or modify a file
3. Stage it with add
4. Review with status
5. Compare with diff (--staged: index vs HEAD)
6. Commit with a message
7. Read the recorded log

Command output is not shown: the supplied documentation does not give exact messages. notes.txt is an example file.

File Staging Workflow

1

Paths are given

loki add takes one or more paths.

2

Blob is stored

Identical bytes produce the same object ID.

3

Index is updated

The path maps to the blob hash.

Working-tree modification

The file changed on disk. Nothing is staged yet; loki diff compares it with the index.

Staged change

The blob exists and the index points to it; loki diff --staged compares the index with HEAD.

Status and Diff

Working tree
files on disk

`loki diff` ⟷

Index
staged: .loki/index

`diff --staged` ⟷

HEAD
last committed state

ILLUSTRATIVE TEXT-LINE DIFF

```
- version = 1
+ version = 2
```

Diff utilities generate and format text-line diffs.

loki status

Displays tracked, staged, modified, deleted and untracked files for the repository.

Commit and History

```
$ loki commit -m "message"
```

1
Staged content
What is in the index.

→

2
Commit created
Tree ID, parent ID(s), author, message.

→

3
Objects stored
Commit, tree and blobs in the object store.

→

4
Commit recorded
In the append-only commits.log.

loki log
Displays the repository's recorded commit log.

Not an ancestry walk
It reads the log file; it does not traverse commits from HEAD.

Branching and Checkout

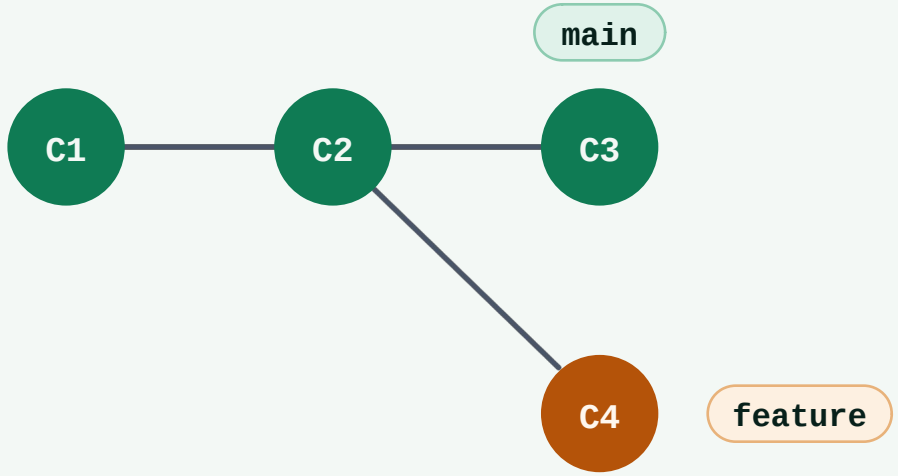
`loki branch`
Lists or creates local branches.

`loki checkout`
Switches the working state to a branch, tag or commit.

Detached HEAD
Checking out a tag or a commit detaches HEAD.

Local only
Branches are files in refs/heads/. There are no remote branches.

ILLUSTRATIVE TWO-BRANCH HISTORY



Branch names and commits are illustrative. Each branch name is a local reference to a commit.

Reset and Revert

`loki reset`

Resets HEAD, the index or the working tree to a target commit or branch state.

`--soft` `--mixed` `--hard`

The mode decides how far the reset reaches beyond HEAD.

`loki revert`

Applies the inverse of a non-merge commit by creating a new commit when successful.

Separate from reset: it adds a commit instead of moving HEAD.

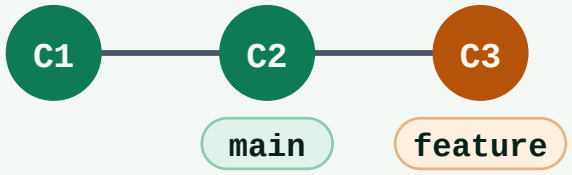
⚠ **Use with care.** Operations that update the working tree can overwrite local changes, and edge cases may differ from Git.

Merge Overview: loki merge <branch>

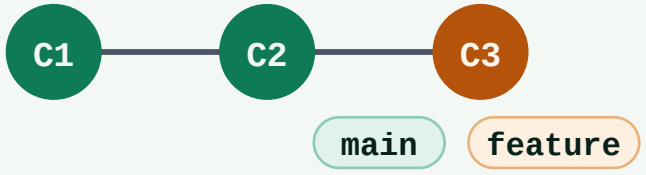
Fast-forward

The branch pointer moves forward; no merge commit is needed.

BEFORE



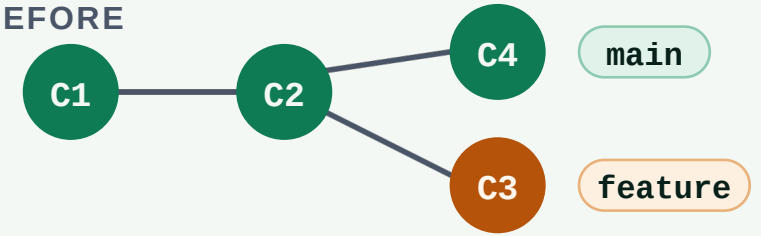
AFTER



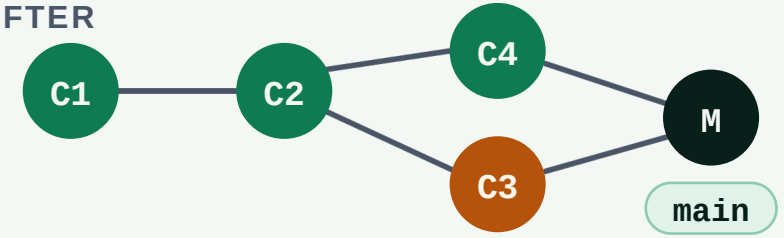
Merge commit

A new commit joins the two lines of work.

BEFORE



AFTER



Fast-forward and 3-way merge flows are implemented. Local branches only. Conflicts are detected at the file/path level (next slides). All names and commits are illustrative.

File-Level Three-Way Merge



For each file path	Outcome
Changed on only one side	That side's version is used
Changed differently on both sides	Reported as a conflict
Unchanged on both sides	The file stays as it is

The documentation lists 3-way merge flows. This table is a conceptual file-level rule; merge decisions are file/path-based, not line-by-line.

Merge Conflicts

ILLUSTRATIVE CONFLICTED FILE

```
<<<<<<< current
greeting = "Hello"
=======
greeting = "Hi"
>>>>>>> other
```

Sample content only; the exact marker format is not specified in the summary.

When it happens

Both branches changed the same file differently. Conflicts are detected at the file/path level.

What Loki leaves

Conflicting files may contain conflict markers.

How to resolve

Edit the files, stage them, then commit.

Merge Workflow

1
Start the merge
`loki merge <branch>`

➡

2
Conflicts reported
Conflicting files are identified.

➡

3
Resolve
Edit the affected files.

➡

4
Stage
`loki add <file>`

➡

5
Complete
`loki commit -m`

No conflicts
The merge can fast-forward, or create a merge commit.

With conflicts
Temporary state files may exist while conflicts are being resolved.

Tags, Ignore Rules and Configuration

Tags

`loki tag [options]`

Create, list, delete or annotate tags. A tag may be a lightweight reference in refs/tags/ or an annotated tag object. Checking out a tag detaches HEAD.

Ignore rules

`.lokiignore`

An optional file at the project root that holds ignore patterns. Its pattern syntax is not specified in the supplied documentation.

Configuration

`loki config [key] [value]`

Reads or writes repository configuration values, stored in .loki/config. internal/config/ also discovers repository roots. Specific keys are not listed in the supplied documentation.

Ignore-pattern examples and configuration keys are left out on purpose rather than guessed.

Error Handling and Testing

What the documentation states

- Merge conflicts are detected at the file/path level.
- Revert applies to non-merge commits only.
- Merge and revert may keep temporary state files while conflicts are resolved.
- Object reads decompress data but do not re-hash it.
- No repository locking or concurrency protocol exists.

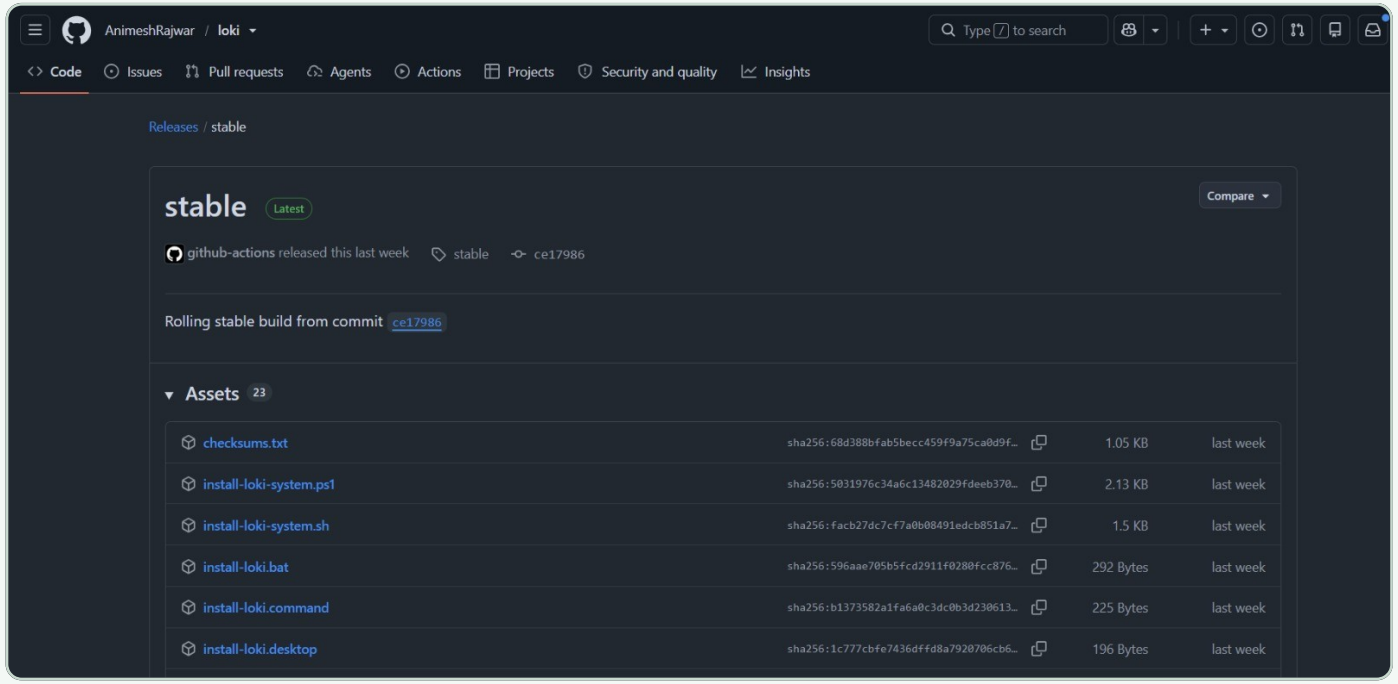
⚠

Not established by the supplied files

- Test counts or coverage percentages
- Continuous-integration results
- Performance benchmarks
- Detailed error messages and validation rules

None of these are reported here.

Latest Stable Release on GitHub



Repository
`AnimeshRajwar/loki`

Release
stable, labelled Latest, built from commit **ce17986**

Assets: 23
checksums.txt and five install files are visible

Installation: Linux, macOS and Windows

Linux and macOS

Same release-installer flow. The scripts detect the platform and download the right binary.

```
# User install (default: $HOME/.local/bin)
curl -fsSL https://raw.githubusercontent.com/AnimeshRajwar/loki/master/scripts/install.sh | bash -s stable

# System install (requires sudo)
curl -fsSL https://raw.githubusercontent.com/AnimeshRajwar/loki/master/scripts/install-system.sh | bash -s stable /usr/local/bin
```

Windows

PowerShell installs the release asset and adds the install folder to your user PATH.

```
powershell -ExecutionPolicy Bypass -Command "Invoke-WebRequest https://raw.githubusercontent.com/AnimeshRajwar/loki/master/scripts/install.ps1 -OutFile install-loki.ps1; .install-loki.ps1 -Channel stable"

# or from PowerShell directly:
# .install-loki.ps1 -Channel latest
```

Verify the install

```
loki --help
```

Release assets (slide 37)

checksums.txt, install-loki-system.ps1 and .sh, install-loki.bat, .command and .desktop

Current Limitations and Future Improvements

DOCUMENTED LIMITATIONS

- Local only: no remote repositories, and no fetch, pull or push.
- loki log reads commits.log; it does not traverse commit ancestry.
- Object reads decompress but do not re-hash to verify IDs.
- No repository locking or concurrency protocol.
- Merges are file/path-based, not line-by-line.
- The index is a custom JSON format, not Git's.

POTENTIAL IMPROVEMENTS: PROPOSALS ONLY

- Remote repository support
- Stronger object integrity verification
- Safer atomic updates and concurrency handling
- More comprehensive automated testing
- Improved documentation and error messages
- Additional merge and history capabilities

None of these are implemented today.

Thank You

PIET NON ASMP – 04

Parul Institute of Engineering and Technology

Parul University, Vadodara